

Lembar Materi & Praktikum: DTO Request-Response, Standardisasi API Response, Pagination, dan Fitur Search

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Praktikum ini melengkapi materi teori dan memberikan panduan implementasi nyata tentang bagaimana merancang API yang siap diproduksi (*production-ready*), efisien, dan memiliki skalabilitas tinggi pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) menggunakan framework **Gin** dan ORM **GORM**. Kita akan belajar memisahkan model database dari *payload* API menggunakan **DTO**, menyusun pembungkus respons JSON yang konsisten, membatasi data kueri menggunakan **Pagination (Limit-Offset)**, serta mengimplementasikan **Fitur Pencarian & Pengurutan** secara aman guna menghindari celah keamanan **SQL Injection**.

Sesi 1: Teori & Desain DTO (Request-Response) serta Standardisasi API Response (45 Menit)

1. Data Transfer Object (DTO)

Dalam arsitektur perangkat lunak modern, kita **sangat tidak dianjurkan** mengekspos model database langsung sebagai representasi input atau output API ke client. Beberapa alasan utamanya meliputi:

- **Keamanan Informasi:** Menyembunyikan kolom database sensitif seperti `deleted_at`, hash password, atau kunci internal lainnya agar tidak dikirim ke klien.
- **Kebebasan Desain (Decoupling):** Jika skema tabel database diubah (misal nama kolom dirubah dari `concert_title` menjadi `title`), kita cukup mengubah pemetaan di dalam DTO tanpa merusak kontrak API yang dikonsumsi oleh frontend.
- **Format Khusus (Data Transformation):** Memudahkan konversi tipe data, misalnya mengubah field bertipe `time.Time` dari database menjadi string terformat `"YYYY-MM-DD"` untuk respons API.

2. Format API Response Konsisten

Frontend engineer akan kesulitan melakukan parsing respons apabila backend mengirimkan format data yang berubah-ubah (misalnya saat sukses data dibungkus objek langsung, namun saat error

data dibungkus array). Oleh karena itu, kita mendesain struktur pembungkus standar (`WebResponse`) untuk setiap tipe endpoint di backend.

Struktur Standard JSON Response:

- **success** (boolean): Indikator utama apakah permintaan berhasil diproses atau tidak.
 - **message** (string): Pesan informatif untuk pengguna atau frontend developer (misal: "Berhasil mengambil data", "Validasi gagal").
 - **data** (any/interface{}): Payload utama data yang dikembalikan. Field ini bernilai `null` atau disembunyikan menggunakan tag `omitempty` jika terjadi error.
 - **meta** (any/interface{}): Objek tambahan untuk menyimpan informasi non-bisnis, seperti metadata pagination (halaman, batas limit, total data).
-

Sesi 2: Teori & Logika Matematis Pagination & Search Filter (45 Menit)

1. Mengapa Butuh Pagination?

Jika sistem tiket konser Anda memiliki 50.000 data konser, memuat seluruh data ke memori dalam sekali panggil (`SELECT * FROM concerts`) akan:

1. Menghabiskan RAM server dan meningkatkan beban I/O database secara drastis.
2. Membuat transfer payload JSON sangat lambat dan meningkatkan latensi jaringan.
3. Menyebabkan aplikasi frontend mengalami *freeze* saat merender ribuan komponen UI.

Dengan **Pagination**, kita membatasi jumlah baris data yang ditarik per halaman menggunakan perintah SQL `LIMIT` dan `OFFSET`.

2. Formula Matematis Offset

Untuk menentukan data ke-berapa yang harus mulai diambil berdasarkan nomor halaman (`page`) dan batas kuantitas data (`limit`), kita menggunakan rumus dasar berikut:
$$\text{Offset} = (\text{Page} - 1) \times \text{Limit}$$

- **Page 1** (Limit = 10): $\text{Offset} = (1 - 1) \times 10 = 0$. (Menampilkan data 1 s.d 10)
- **Page 2** (Limit = 10): $\text{Offset} = (2 - 1) \times 10 = 10$. (Melewati 10 data pertama, menampilkan data 11 s.d 20)

3. Kueri Total Data (Double Querying)

Untuk memberikan visual tombol halaman pada frontend (misal: tombol 1, 2, 3... `Next`), backend wajib mengirimkan total data keseluruhan *sebelum* dikenakan filter `limit` dan `offset`. Sehingga

dilakukan dua tahap eksekusi kueri:

1. **Kueri 1:** Menghitung jumlah data keseluruhan (`SELECT COUNT(*) FROM concerts WHERE ...`).
2. **Kueri 2:** Mengambil data spesifik ter-paginasi (`SELECT * FROM concerts WHERE ... LIMIT 10 OFFSET 0`).

4. Mitigasi SQL Injection pada Fitur Pencarian

Ketika membuat fitur pencarian berbasis teks (`search`), developer sering terjebak menyusun kueri SQL menggunakan teknik penggabungan string langsung (string concatenation):

```
// ❌ SANGAT BERBAHAYA - RENTAN TERHADAP SQL INJECTION!  
db.Where("title LIKE '%" + searchInput + "%'")
```

Jika penyerang mengirimkan input bermuatan SQL jahat seperti `' OR '1'='1'`, mereka dapat memanipulasi kueri database untuk mencuri seluruh data.

Strategi Pertahanan: Selalu gunakan parameterized queries di GORM menggunakan placeholder tanda tanya (`?`). GORM akan mengesampingkan karakter khusus berbahaya secara otomatis (*escaping*):

```
// ✅ AMAN DARI SQL INJECTION  
db.Where("title ILIKE ? OR venue ILIKE ?", "%"+searchInput+"%", "%"+searchInput+"%")
```

Sesi 3: Laboratorium Praktikum Mandiri – Pembuatan DTO & Standard Response Wrapper (45 Menit)

Lakukan pembuatan dan pembaruan file-file DTO berikut pada direktori proyek `go-tiket-konser` Anda.

1. File DTO Response Standar: [dto/response.go](#)

Buat berkas baru bernama `response.go` di dalam folder `dto/` untuk menyimpan definisi response wrapper terstandarisasi:

```
package dto  
  
// WebResponse adalah format wrapper standar untuk API response  
type WebResponse struct {  
    Success bool    `json:"success"`  
    Message string  `json:"message"`  
    Data    interface{} `json:"data,omitempty"`  
}
```

```

    Meta    interface{} `json:"meta,omitempty"`
}

// PaginationMeta mendefinisikan metadata pagination untuk respons bertipe list/array
type PaginationMeta struct {
    Page      int    `json:"page"`
    Limit     int    `json:"limit"`
    TotalData int64  `json:"total_data"`
    TotalPage int    `json:"total_page"`
}

```

2. Update File DTO Concert: [dto/concert.go](#)

Buka file `dto/concert.go` dan tambahkan struct `ConcertQueryRequest` di bagian bawah berkas untuk menampung parameter kueri dari URL (*query parameters*):

```

package dto

import "time"

type ConcertRequest struct {
    Title      string `json:"title" binding:"required,min=5,max=150"`
    Description string `json:"description" binding:"max=255"`
    Date       string `json:"date" binding:"required,future_date"`
    Venue      string `json:"venue" binding:"required,min=3,max=100"`
    Status     string `json:"status" binding:"required,oneof=active upcoming canceled"`
}

type ConcertResponse struct {
    ID          int       `json:"id"`
    Title       string    `json:"title"`
    Description  string    `json:"description"`
    Date        string    `json:"date"`
    Venue       string    `json:"venue"`
    Status      string    `json:"status"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

+
+// ConcertQueryRequest menangkap query parameter untuk pencarian, pagination, dan
+// sorting
+type ConcertQueryRequest struct {
+    Page      int    `form:"page" binding:"omitempty,numeric,gte=1"`
+    Limit     int    `form:"limit" binding:"omitempty,numeric,gte=1"`
+    Search     string `form:"search"`
+    Sort      string `form:"sort" binding:"omitempty,oneof=date_asc date_desc title_asc title_desc"`
+}

```

Sesi 4: Laboratorium Praktikum Mandiri – Integrasi Repository, Service, dan Handler (45 Menit)

Kita akan mengintegrasikan kueri pencarian, pagination, dan pengurutan dinamis ini ke dalam arsitektur terdekopel di modul Concert.

1. Update Repository Concert: [repository/concert.go](#)

Buka berkas `repository/concert.go`, lalu tambahkan method `FindAllPaginated` ke dalam interface dan struct implementasinya:

```
package repository

import (
    "go-tiket-konser/models"

    "gorm.io/gorm"
)

type ConcertRepository interface {
    Create(concert *models.Concert) error
    FindAll() ([]models.Concert, error)
    // Tambahkan method baru untuk pencarian, pagination, dan sorting
    FindAllPaginated(search string, limit int, offset int, sort string)
    ([]models.Concert, int64, error)
    FindByID(id int) (models.Concert, error)
    Update(concert *models.Concert) error
    Delete(id int) error
}

type concertRepository struct {
    db *gorm.DB
}

func NewConcertRepository(db *gorm.DB) ConcertRepository {
    return &concertRepository{db: db}
}

// ... implementasi method Create, FindAll, FindByID, Update, dan Delete dipertahankan
...

// FindAllPaginated memproses kueri GORM dengan pencarian LIKE, hitung total data,
// pengurutan, dan limit-offset
func (r *concertRepository) FindAllPaginated(search string, limit int, offset int,
sort string) ([]models.Concert, int64, error) {
    var concerts []models.Concert
    var total int64

    // Inisialisasi model query GORM
```

```

    query := r.db.Model(&models.Concert{})

    // 1. Terapkan Filter Pencarian (Search) jika parameter search dikirim oleh
    user
    if search != "" {
        // Menggunakan placeholder ? untuk mencegah celah SQL Injection
        query = query.Where("title ILIKE ? OR venue ILIKE ?", "%"+search+"%",
"%"+search+"%")
    }

    // 2. Hitung Total Data yang Cocok Sebelum Limit dan Offset (Double Querying
    Step 1)
    if err := query.Count(&total).Error; err != nil {
        return nil, 0, err
    }

    // 3. Terapkan Klausa Pengurutan (Sorting)
    switch sort {
    case "date_asc":
        query = query.Order("date ASC")
    case "date_desc":
        query = query.Order("date DESC")
    case "title_asc":
        query = query.Order("title ASC")
    case "title_desc":
        query = query.Order("title DESC")
    default:
        query = query.Order("id DESC") // Pengurutan default jika tidak
    ditentukan
    }

    // 4. Batasi Data dengan Limit dan Lewati Data dengan Offset (Double Querying
    Step 2)
    err := query.Limit(limit).Offset(offset).Find(&concerts).Error

    return concerts, total, err
}

```

2. Update Service Concert: [service/concert.go](#)

Buka berkas `service/concert.go`. Kita perlu memperbarui method `GetAllConcerts` agar menerima parameter DTO kueri dan mengembalikan response bertipe DTO dengan metadata pagination:

```

package service

import (
    "math"
    "go-tiket-konser/dto"
    "go-tiket-konser/models"

```

```

        "go-tiket-konser/repository"
    )

    type ConcertService interface {
        CreateConcert(concert *models.Concert) error
        // Ubah method ini agar menerima Query Request DTO dan memulangkan respons DTO
        // ber-Pagination
        GetAllConcerts(req dto.ConcertQueryRequest) ([]dto.ConcertResponse,
        dto.PaginationMeta, error)
        GetConcertByID(id int) (models.Concert, error)
        UpdateConcert(concert *models.Concert) error
        DeleteConcert(id int) error
    }

    type concertService struct {
        repo repository.ConcertRepository
    }

    func NewConcertService(repo repository.ConcertRepository) ConcertService {
        return &concertService{repo: repo}
    }

    // ... implementasi method CreateConcert, GetConcertByID, UpdateConcert, dan
    // DeleteConcert tetap dipertahankan ...

    func (s *concertService) GetAllConcerts(req dto.ConcertQueryRequest)
    ([]dto.ConcertResponse, dto.PaginationMeta, error) {
        // 1. Validasi & Set Nilai Default untuk Limit
        limit := req.Limit
        if limit ≤ 0 {
            limit = 10 // Default limit per halaman
        }
        if limit > 100 {
            limit = 100 // Batasan keamanan maksimum limit demi memitigasi
serangan DOS
        }

        // 2. Validasi & Set Nilai Default untuk Page
        page := req.Page
        if page ≤ 0 {
            page = 1 // Default halaman pertama
        }

        // 3. Hitung Kalkulasi Offset
        offset := (page - 1) * limit

        // 4. Panggil Kueri ke Repositori
        concerts, totalData, err := s.repo.FindAllPaginated(req.Search, limit, offset,
req.Sort)
        if err ≠ nil {
            return nil, dto.PaginationMeta{}, err
        }
    }

```

```

// 5. Transformasi Entity Model Database ke dalam bentuk DTO Response
var responses []dto.ConcertResponse
for _, c := range concerts {
    responses = append(responses, dto.ConcertResponse{
        ID:          c.ID,
        Title:       c.Title,
        Description: c.Description,
        Date:       c.Date.Format("2006-01-02"),
        Venue:      c.Venue,
        Status:     c.Status,
        CreatedAt:  c.CreatedAt,
        UpdatedAt:  c.UpdatedAt,
    })
}

// 6. Hitung Total Halaman (Membulatkan ke Atas)
totalPage := 0
if totalData > 0 {
    totalPage = int(math.Ceil(float64(totalData) / float64(limit)))
}

// 7. Bentuk metadata pagination
meta := dto.PaginationMeta{
    Page:      page,
    Limit:     limit,
    TotalData: totalData,
    TotalPage: totalPage,
}

return responses, meta, nil
}

```

3. Update Handler Concert: [handler/concert.go](#)

Buka berkas `handler/concert.go` dan perbarui method `GetConcerts` agar menangkap query string dari URL, menguji validasinya, lalu mengembalikan respons sukses bertipe `dto.WebResponse` yang konsisten:

```

package handler

import (
    "errors"
    "net/http"
    "strconv"
    "time"

    "go-tiket-konser/dto"
    "go-tiket-konser/models"
    "go-tiket-konser/service"

```



```

        "github.com/gin-gonic/gin"
        "gorm.io/gorm"
    )

    // ... constructor ConcertHandler dan method CreateConcert, GetConcertByID,
    UpdateConcert, dan DeleteConcert tetap ada ...

    // GetConcerts handles GET /api/v1/concerts dengan pencarian, pagination, dan
    standardisasi respons
    func (h *ConcertHandler) GetConcerts(c *gin.Context) {
        var req dto.ConcertQueryRequest

        // 1. Bind parameter URL Query ke DTO kustom (c.ShouldBindQuery mengikat
        parameter bertag `form`)
        if err := c.ShouldBindQuery(&req); err != nil {
            c.JSON(http.StatusBadRequest, dto.WebResponse{
                Success: false,
                Message: "Validasi parameter pencarian gagal",
                Data:    err.Error(),
            })
            return
        }

        // 2. Panggil layer service untuk memproses bisnis pencarian & pagination
        concerts, meta, err := h.service.GetAllConcerts(req)
        if err != nil {
            c.JSON(http.StatusInternalServerError, dto.WebResponse{
                Success: false,
                Message: "Gagal mengambil daftar konser",
                Data:    err.Error(),
            })
            return
        }

        // 3. Kembalikan WebResponse terstandarisasi dengan meta pagination
        c.JSON(http.StatusOK, dto.WebResponse{
            Success: true,
            Message: "Berhasil menampilkan daftar konser",
            Data:    concerts,
            Meta:    meta,
        })
    }
}

```

4. Uji Coba Integrasi API menggunakan Postman atau cURL

Jalankan server aplikasi Anda dengan perintah `go run main.go`, lalu uji coba endpoint Anda menggunakan skenario query string berikut:

1. **Pencarian & Pengurutan:** Menampilkan konser yang bertempat di "Jakarta" diurutkan dari tanggal paling awal.

```
GET http://localhost:8080/api/v1/concerts?search=Jakarta&sort=date_asc
```

2. **Pagination Halaman Pertama:** Membatasi output maksimal hanya 2 data per halaman.

```
GET http://localhost:8080/api/v1/concerts?page=1&limit=2
```

3. **Pagination Halaman Kedua:** Melihat data urutan 3 s.d 4.

```
GET http://localhost:8080/api/v1/concerts?page=2&limit=2
```

Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri) (45 Menit)

Selesaikan tantangan berikut secara mandiri untuk memperkuat kompetensi dalam perancangan API dinamis, bersih, dan aman.

Tantangan 1: Implementasi Pagination & Search pada Modul Booking

Administrator dan staf membutuhkan menu peninjauan seluruh riwayat booking tiket. Saat data semakin membengkak, endpoint `GET /api/v1/bookings` harus dilindungi dengan pagination.

- **Tugas:**

1. Buatlah struct `BookingQueryRequest` di dalam berkas DTO untuk menampung query parameter `page`, `limit`, `search`, dan `sort`.
2. Sesuaikan kueri pada `repository/booking.go` dan `service/booking.go`.
3. Parameter `search` harus mencari data secara parsial berdasarkan `booking_code` atau nama customer (`customer.name` dengan memanfaatkan kueri JOIN database).
4. Batasi akses API hanya bagi role `ADMIN` dan `STAFF` menggunakan middleware otorisasi yang telah dibuat pada hari sebelumnya.
5. Kembalikan data terbungkus dalam format `WebResponse` dengan meta pagination.

Tantangan 2: Fitur Filter Kategori Tiket Berdasarkan Range Harga

Di bagian halaman pembelian tiket untuk customer, customer membutuhkan penyaringan kategori tiket konser berdasarkan harga terendah (`min_price`) dan harga tertinggi (`max_price`).

- **Tugas:**

1. Buka berkas `dto/ticket_category.go`, tambahkan struct kueri pencarian kategori tiket baru yang memuat field `MinPrice` dan `MaxPrice` (tipe `float64` atau `int`).
2. Modifikasi `repository/ticket_category.go` agar mengaplikasikan klausa filter dinamis:

```
if minPrice > 0 {  
    query = query.Where("price ≥ ?", minPrice)  
}  
if maxPrice > 0 {  
    query = query.Where("price ≤ ?", maxPrice)  
}
```

3. Salurkan input dari layer Handler melalui Service ke Repository.
4. Pastikan data keluaran tetap ter-paginasi dan dibungkus menggunakan standard response wrapper.